

Практическое занятие 32: Работа с сериализаторами и представлениями API

Цель занятия:

- Углубить знания по работе с Django REST Framework (DRF).
 - Научиться создавать сложные сериализаторы для вложенных данных и связанных моделей.
 - Освоить использование различных видов представлений (Views) в DRF, включая ViewSets и Generic Views.
 - Реализовать функциональность API для создания, чтения, обновления и удаления (CRUD) данных.
 - Понять, как добавить фильтрацию, поиск и пагинацию в API.
-

Задачи занятия

1. Повторение основ DRF и сериализаторов
 2. Создание вложенных сериализаторов для моделей `Event` и `Review`
 3. Использование ViewSets и маршрутизаторов
 4. Реализация фильтрации, поиска и пагинации
 5. Настройка разрешений и аутентификации
 6. Тестирование API с использованием Postman или другого инструмента
-

Задача 1: Повторение основ DRF и сериализаторов

1.1. Что такое сериализаторы в DRF?

- **Сериализаторы** в DRF позволяют преобразовывать сложные типы данных, такие как объекты моделей Django, в простые типы данных Python, которые могут быть легко преобразованы в формат JSON или XML.
- **Десериализация**: Преобразование входящих данных обратно в сложные типы, после проверки данных на корректность.

1.2. Типы представлений в DRF

- **Function-Based Views (FBV)**: Представления, основанные на функциях, с использованием декораторов `@api_view`.

- **Class-Based Views (CBV):** Представления, основанные на классах, используя `APIView`, `GenericAPIView` и их производные.
- **ViewSet:** Позволяют объединить логически связанные представления в один класс, облегчая маршрутизацию.

Задача 2: Создание вложенных сериализаторов для моделей `Event` и `Review`

2.1. Создание сериализатора для модели `Review`

Шаги:

1. Откройте `serializers.py` в приложении `events` и добавьте `ReviewSerializer`:

```
# events/serializers.py

from rest_framework import serializers
from .models import Event, Review

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ['id', 'name', 'email', 'rating', 'comment',
                  'created_at']
```

2. Обновите `EventSerializer`, чтобы включить связанные отзывы:

```
# events/serializers.py

class EventSerializer(serializers.ModelSerializer):
    reviews = ReviewSerializer(many=True, read_only=True)

    class Meta:
        model = Event
        fields = ['id', 'title', 'description', 'date', 'location',
                  'reviews']
```

Пояснение:

- Добавляем поле `reviews` в `EventSerializer`, которое использует `ReviewSerializer` для отображения связанных отзывов.
- Устанавливаем `many=True`, так как мероприятие может иметь несколько отзывов.

- Устанавливаем `read_only=True`, чтобы предотвратить создание отзывов через сериализатор мероприятия.

2.2. Создание сериализатора для создания и обновления отзывов

Шаги:

1. Создайте `ReviewCreateSerializer` для обработки создания отзывов:

```
# events/serializers.py

class ReviewCreateSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ['id', 'name', 'email', 'rating', 'comment', 'event']

    def create(self, validated_data):
        return Review.objects.create(**validated_data)
```

Пояснение:

- Этот сериализатор будет использоваться для создания новых отзывов.
- Включаем поле `event`, чтобы связать отзыв с мероприятием.

2.3. Обновление моделей для корректной работы с сериализаторами

Убедитесь, что в модели `Event` есть обратная связь с отзывами:

```
# events/models.py

class Event(models.Model):
    # ... существующие поля ...
    # Отзывы связаны через related_name='reviews' в модели Review
```

Пояснение:

- Если в модели `Review` вы не указали `related_name`, Django по умолчанию создаст связь `review_set`.
- Для удобства лучше явно указать `related_name` в поле `ForeignKey` модели `Review`:

```
# events/models.py

class Review(models.Model):
```

```
# ... существующие поля ...
event = models.ForeignKey(Event, related_name='reviews',
on_delete=models.CASCADE)
```

Не забудьте выполнить миграции после изменений в моделях:

```
python manage.py makemigrations
python manage.py migrate
```

Задача 3: Использование ViewSets и маршрутизаторов

3.1. Создание ViewSet для модели Event

Шаги:

1. Откройте `views.py` и создайте `EventViewSet`:

```
# events/views.py

from rest_framework import viewsets
from .models import Event
from .serializers import EventSerializer

class EventViewSet(viewsets.ModelViewSet):
    queryset = Event.objects.all()
    serializer_class = EventSerializer
```

2. Создайте `ReviewViewSet` для модели `Review`:

```
# events/views.py

from .models import Review
from .serializers import ReviewSerializer, ReviewCreateSerializer

class ReviewViewSet(viewsets.ModelViewSet):
    queryset = Review.objects.all()
    serializer_class = ReviewSerializer

    def get_serializer_class(self):
        if self.action in ['create', 'update', 'partial_update']:
```

```
        return ReviewCreateSerializer
    return ReviewSerializer
```

Пояснение:

- Используем `get_serializer_class`, чтобы использовать разные сериализаторы в зависимости от действия.

3.2. Настройка маршрутизатора для ViewSets

Шаги:

1. Откройте `urls.py` в приложении `events`:

```
# events/urls.py

from rest_framework import routers
from .views import EventViewSet, ReviewViewSet

router = routers.DefaultRouter()
router.register(r'api/events', EventViewSet)
router.register(r'api/reviews', ReviewViewSet)
```

2. Добавьте маршруты маршрутизатора в `urlpatterns`:

```
# events/urls.py

urlpatterns = [
    # ... ваши другие URL-маршруты ...
] + router.urls
```

Пояснение:

- Маршрутизатор автоматически создаст необходимые URL для ViewSets.

Задача 4: Реализация фильтрации, поиска и пагинации

4.1. Добавление пагинации

Шаги:

1. Настройте пагинацию в `settings.py`:

```
# settings.py

REST_FRAMEWORK = {
    # ... ваши другие настройки ...
    'DEFAULT_PAGINATION_CLASS':
    'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 10,
}
```

Пояснение:

- Устанавливаем размер страницы по умолчанию на 10 элементов.

4.2. Добавление фильтрации и поиска

Шаги:

1. Установите пакет `django-filter`:

```
pip install django-filter
```

2. Добавьте `django_filters` в `INSTALLED_APPS` в `settings.py`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'django_filters',
]
```

3. Настройте фильтрацию в `settings.py`:

```
# settings.py

REST_FRAMEWORK = {
    # ... ваши другие настройки ...
    'DEFAULT_FILTER_BACKENDS':
    ['django_filters.rest_framework.DjangoFilterBackend',
    'rest_framework.filters.SearchFilter'],
}
```

4. Добавьте фильтрацию и поиск в `EventViewSet`:

```
# events/views.py

class EventViewSet(viewsets.ModelViewSet):
```

```
queryset = Event.objects.all()
serializer_class = EventSerializer
filterset_fields = ['date', 'location']
search_fields = ['title', 'description']
```

Пояснение:

- `filterset_fields` позволяет фильтровать по указанным полям.
- `search_fields` позволяет выполнять поиск по указанным полям.

5. Добавьте фильтрацию и поиск в `ReviewViewSet` (если необходимо):

```
# events/views.py

class ReviewViewSet(viewsets.ModelViewSet):
    queryset = Review.objects.all()
    # ... остальной код ...
    filterset_fields = ['rating', 'event']
    search_fields = ['name', 'comment']
```

4.3. Тестирование фильтрации и поиска

- Примеры запросов:
 - Фильтрация по дате:

```
GET /api/events/?date=2023-10-31
```

- Поиск по заголовку:

```
GET /api/events/?search=концерт
```

- Комбинированная фильтрация и пагинация:

```
GET /api/events/?date=2023-10-31&search=концерт&page=2
```

Задача 5: Настройка разрешений и аутентификации

5.1. Использование разрешений

Шаги:

1. Импортируйте класс разрешений в `views.py` :

```
# events/views.py

from rest_framework.permissions import IsAuthenticatedOrReadOnly
```

2. Примените разрешения к ViewSets:

```
# events/views.py

class EventViewSet(viewsets.ModelViewSet):
    # ... ваш существующий код ...
    permission_classes = [IsAuthenticatedOrReadOnly]
```

Пояснение:

- `IsAuthenticatedOrReadOnly` позволяет аутентифицированным пользователям создавать и изменять данные, а неаутентифицированным — только читать.

3. Примените аналогичные разрешения к `ReviewViewSet` :

```
# events/views.py

class ReviewViewSet(viewsets.ModelViewSet):
    # ... ваш существующий код ...
    permission_classes = [IsAuthenticatedOrReadOnly]
```

5.2. Настройка аутентификации через токены

Шаги:

1. Установите `django-rest-framework-simplejwt` :

```
pip install django-rest-framework-simplejwt
```

2. Настройте аутентификацию в `settings.py` :

```
# settings.py

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    # ... остальные настройки ...
}
```

3. Добавьте маршруты для получения токена в `urls.py`:

```
# events/urls.py

from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns += [
    path('api/token/', TokenObtainPairView.as_view(),
         name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(),
         name='token_refresh'),
]
```

Пояснение:

- `TokenObtainPairView` используется для получения токена доступа и обновления.
- `TokenRefreshView` используется для обновления токена доступа.

4. Тестирование аутентификации:

- **Получение токена:**

```
POST /api/token/
{
    "username": "your_username",
    "password": "your_password"
}
```

- **Использование токена при запросах:**

Добавьте заголовок `Authorization: Bearer your_access_token` к вашим запросам.

Задача 6: Тестирование API с использованием Postman или другого инструмента

6.1. Использование Postman для тестирования

- **Шаги:**
 - Откройте Postman и создайте новый запрос.
 - Установите метод запроса (GET, POST, PUT, DELETE).

- Введите URL эндпоинта API.
- Если требуется аутентификация, добавьте заголовок `Authorization` с токеном.
- Для методов `POST` и `PUT` добавьте тело запроса в формате JSON.
- Отправьте запрос и проанализируйте ответ.

6.2. Проверка функциональности

- Проверьте следующие сценарии:
 - Получение списка мероприятий с пагинацией, фильтрацией и поиском.
 - Получение деталей конкретного мероприятия.
 - Создание нового мероприятия (для аутентифицированных пользователей).
 - Обновление и удаление мероприятия (для аутентифицированных пользователей).
 - Работа с отзывами: получение списка, создание, обновление, удаление.
-

Дополнительные задания

1. Реализация вложенных маршрутов для отзывов мероприятия

Цель: Создать эндпоинты для работы с отзывами, привязанными к конкретному мероприятию.

Шаги:

1. Установите пакет `drf-nested-routers` :

```
pip install drf-nested-routers
```

2. Настройте вложенные маршруты в `urls.py` :

```
# events/urls.py

from rest_framework_nested import routers
from .views import EventViewSet, ReviewViewSet

router = routers.DefaultRouter()
router.register(r'api/events', EventViewSet)

events_router = routers.NestedDefaultRouter(router, r'api/events',
lookup='event')
```

```
events_router.register(r'reviews', ReviewViewSet, basename='event-
reviews')

urlpatterns = [
    # ... другие URL ...
] + router.urls + events_router.urls
```

3. Обновите ReviewViewSet для работы с вложенными маршрутами:

```
# events/views.py

class ReviewViewSet(viewsets.ModelViewSet):
    serializer_class = ReviewSerializer

    def get_queryset(self):
        return Review.objects.filter(event_id=self.kwargs['event_pk'])

    def perform_create(self, serializer):
        event = get_object_or_404(Event, pk=self.kwargs['event_pk'])
        serializer.save(event=event)
```

Пояснение:

- Теперь отзывы будут доступны по URL: `/api/events/{event_id}/reviews/`.

2. Добавление документации API с помощью Swagger или Redoc

Цель: Автоматически сгенерировать документацию для вашего API.

Шаги:

1. Установите пакет drf-yasg :

```
pip install drf-yasg
```

2. Добавьте его в INSTALLED_APPS в settings.py :

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'drf_yasg',
]
```

3. Настройте маршруты для документации в urls.py :

```
# events/urls.py

from django.urls import re_path
from rest_framework import permissions
from drf_yasg.views import get_schema_view
from drf_yasg import openapi

schema_view = get_schema_view(
    openapi.Info(
        title="Event API",
        default_version='v1',
        description="API для управления мероприятиями и отзывами",
    ),
    public=True,
    permission_classes=(permissions.AllowAny,),
)

urlpatterns += [
    re_path(r'^swagger(?P<format>\.json|\.yaml)$',
    schema_view.without_ui(cache_timeout=0), name='schema-json'),
    re_path(r'^swagger/$', schema_view.with_ui('swagger',
    cache_timeout=0), name='schema-swagger-ui'),
    re_path(r'^redoc/$', schema_view.with_ui('redoc', cache_timeout=0),
    name='schema-redoc'),
]
```

4. Проверьте доступность документации:

- Swagger UI: <http://127.0.0.1:8000/swagger/>
- Redoc: <http://127.0.0.1:8000/redoc/>

Заключение

Поздравляем! Вы успешно:

- Создали сложные сериализаторы для вложенных и связанных моделей.
- Использовали ViewSets и маршрутизаторы для упрощения кода и маршрутизации.
- Реализовали фильтрацию, поиск и пагинацию в вашем API.
- Настроили разрешения и аутентификацию для защиты API.
- Тестировали ваш API с помощью инструментов, таких как Postman.
- Изучили дополнительные возможности DRF для создания мощного и гибкого API.

Если у вас возникнут дополнительные вопросы или потребуется помощь с каким-либо шагом — обращайтесь!

Успехов в разработке!

Вопросы для обсуждения

- **Почему сериализаторы важны в Django REST Framework?**
 - Сериализаторы позволяют преобразовывать сложные типы данных, такие как модели Django, в форматы данных, пригодные для передачи по сети (например, JSON), и обратно. Это обеспечивает эффективную и безопасную передачу данных между клиентом и сервером, а также валидацию входящих данных. Без сериализаторов было бы сложно управлять преобразованием и проверкой данных, что могло бы привести к ошибкам и уязвимостям в приложении.
 - **В чем разница между ViewSets и Generic Views в DRF, и когда использовать каждый из них?**
 - ViewSets объединяют несколько представлений (например, list, create, retrieve, update, destroy) в одном классе, упрощая маршрутизацию и уменьшая количество кода. Это особенно полезно для стандартных CRUD операций, позволяя быстро создавать API без необходимости писать отдельные представления для каждого действия. Generic Views предоставляют более гибкий подход для создания отдельных представлений с определенной функциональностью, что полезно, когда требуется более детальная настройка или нестандартное поведение. Использовать ViewSets рекомендуется для стандартных API, а Generic Views — когда необходима более тонкая настройка или специфические действия.
 - **Как реализовать фильтрацию и пагинацию в Django REST Framework?**
 - Для реализации фильтрации данных можно использовать пакет django-filter вместе с DRF. Необходимо установить django-filter, добавить его в INSTALLED_APPS и настроить DEFAULT_FILTER_BACKENDS в settings.py. Затем в ViewSet определить поля, по которым будет производиться фильтрация, используя filterset_fields. Для пагинации нужно настроить параметры пагинатора в settings.py, указав DEFAULT_PAGINATION_CLASS и PAGE_SIZE. DRF автоматически применит эти настройки к API-эндпоинтам, обеспечивая удобную навигацию по большим наборам данных.
-

Полезные ресурсы

- [Документация Django REST Framework](#)
- [Документация по сериализаторам DRF](#)

- [Документация по ViewSets и маршрутизаторам](#)
- [drf-nested-routers](#)
- [drf-yasg: Генерация документации Swagger](#)